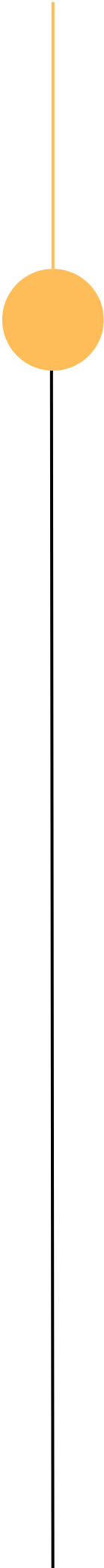


**GEETANJALI
DHOKE**

BUSINESS ANALYST



API Documentation

Document Information

Field	Value
Project Name	
Document Name	API Documentation
Version	1
Author	
Reviewer	
Date	
Status	Draft / Approved

Revision History

Version	Date	Author	Changes
1			Initial Draft

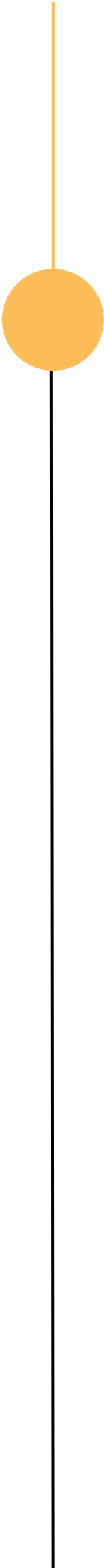


Table of Contents

1. Introduction
2. Base URL
3. Authentication
4. Common Headers
5. API Standards
6. Endpoints
7. Error Handling
8. Rate Limiting
9. Webhooks (Optional)
10. API Versioning
11. Changelog
12. Appendix

1. Introduction

Purpose

Describe the purpose of the API.

Example:

This API provides endpoints for managing users, appointments, billing, and reports for the Clinic Management System.

Supported Formats

- JSON
- REST
- HTTPS

Base URL

Development

<https://dev-api.example.com/v1>

Staging

<https://staging-api.example.com/v1>

Production

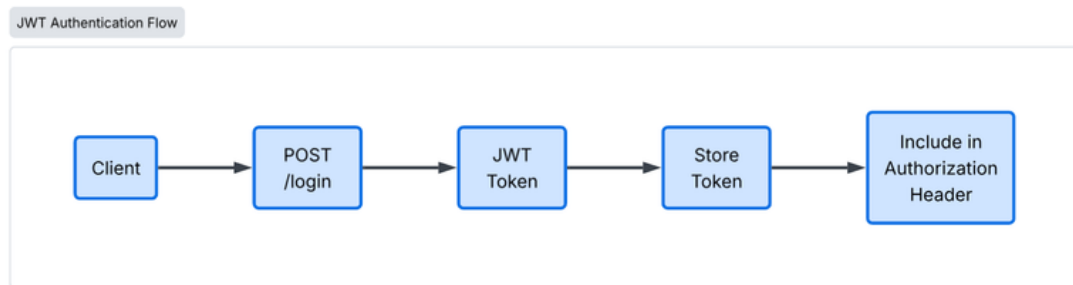
<https://api.example.com/v1>

2. Authentication

Authentication Type

- JWT Bearer Token
- OAuth 2.0
- API Key (choose as applicable)

Login Flow

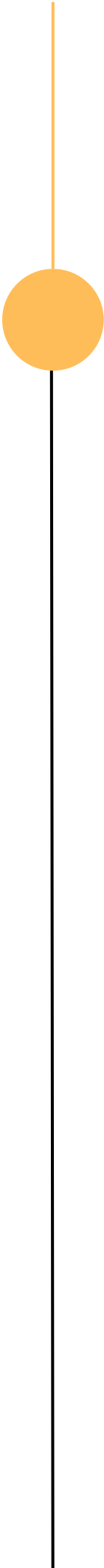


Example Header

Authorization: Bearer <jwt_token>

3. Common Headers

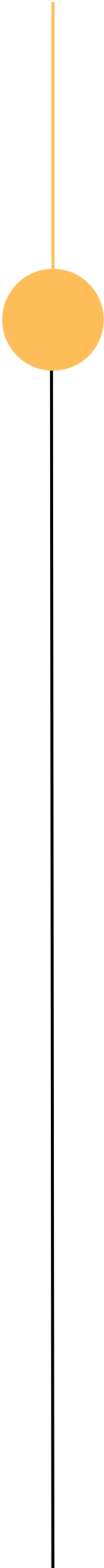
Header	Required	Description
Authorization	Yes	JWT token
Content-Type	Yes	application/json
Accept	Yes	application/json



4. API Standards

HTTP Methods

Method	Purpose
GET	Retrieve data
POST	Create resource
PUT	Replace resource
PATCH	Update resource
DELETE	Delete resource



HTTP Status Codes

Code	Meaning
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
422	Validation Error
500	Internal Server Error

5. API Endpoints

Endpoint: User Login

Endpoint

POST /auth/login

Description

Authenticate a user and return an access token.

Authentication

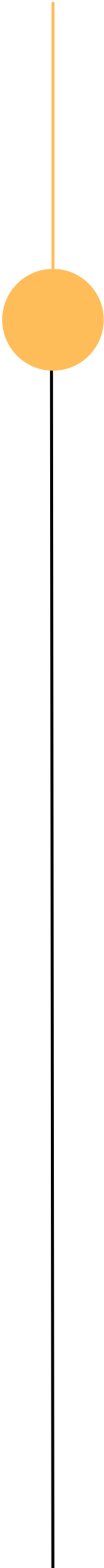
None

Request Body

```
{  
  "email": "john@example.com",  
  "password": "Password@123"  
}
```

Success Response (200)

```
{  
  "success": true,  
  "message": "Login successful",  
  "data": {  
    "accessToken": "jwt-token",  
    "refreshToken": "refresh-token",  
    "user": {  
      "id": "USR001",  
      "name": "John Doe",  
      "role": "Admin"  
    }  
  }  
}
```



Validation Rules

Field	Validation
Email	Required, valid email
Password	Required

Error Responses

Invalid Credentials (401)

```
{  
  "success": false,  
  "message": "Invalid email or password"  
}
```

Endpoint: Get User Profile

Endpoint

GET /users/profile

Authentication

JWT Required

Path Parameters

None

Query Parameters

None

Success Response

```
{
  "success": true,
  "data": {
    "id": "USR001",
    "name": "John Doe",
    "email": "john@example.com"
  }
}
```

Endpoint: Update User

Endpoint

PUT /users/{id}

Path Parameters

Parameter	Type	Required
id	UUID	Yes

Request Body

```
{
  "name": "John Doe",
  "phone": "9876543210"
}
```

Success Response

```
{
  "success": true,
  "message": "User updated successfully"
}
```

6. Request Parameters

Path Parameters

Parameter	Type	Description
id	UUID	Resource identifier

Query Parameters

Parameter	Type	Description
page	Integer	Page number
limit	Integer	Number of records
search	String	Search keyword
sort	String	Sort field

Request Body

Document every request field.

Field	Type	Required	Description
name	String	Yes	Full name
email	String	Yes	Email address
phone	String	No	Mobile number

7. Response Structure

Standard Success Response

```
{  
  "success": true,  
  "message": "Operation completed successfully",  
  "data": {}  
}
```

Standard Error Response

```
{  
  "success": false,  
  "message": "Validation failed",  
  "errors": [  
    {  
      "field": "email",  
      "message": "Email is required"  
    }  
  ]  
}
```

8. Error Codes

Code	Description
AUTH001	Invalid credentials
AUTH002	Token expired
USER001	User not found
VALID001	Validation failed
SERVER001	Internal server error

9. Pagination

Request

GET /users?page=1&limit=20

Response

```
{
  "data": [],
  "pagination": {
    "page": 1,
    "limit": 20,
    "totalRecords": 250,
    "totalPages": 13
  }
}
```

10. Filtering & Sorting

Example

GET /appointments?doctorId=123&status=confirmed&sort=date

11. Rate Limiting

Endpoint	Limit
Login	5 requests/minute
General APIs	100 requests/minute
Public APIs	50 requests/minute

12. Webhooks (Optional)

Event

Appointment Created

Payload

```
{  
  "event": "appointment.created",  
  "appointmentId": "APT001",  
  "timestamp": "2026-07-26T10:00:00Z"  
}
```

13. API Versioning

Current Version

/v1

Future Versions

/v2

Versioning Strategy

- URI Versioning (/v1)
- Header Versioning (optional)

14. Security

- HTTPS only
- JWT Authentication
- Refresh Tokens
- Rate Limiting
- Input Validation
- SQL Injection Prevention
- XSS Protection
- CORS Configuration

15. Testing

Postman Collection

(Add link)

Swagger/OpenAPI

(Add link)

16. Changelog

Version	Date	Changes
1		Initial Release
1.1		Added Appointment APIs

17. Appendix

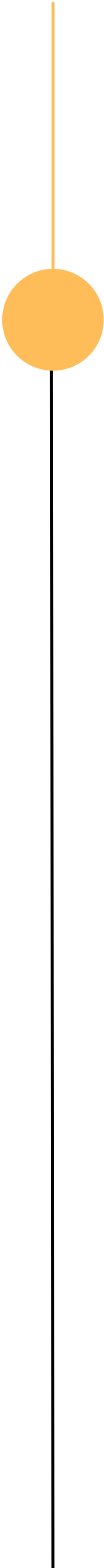
API Naming Conventions

- Use nouns instead of verbs.
- Use plural resource names (/users, /appointments).
- Use lowercase URLs.
- Use hyphens for multi-word resources (/patient-records).

Folder Structure

/api

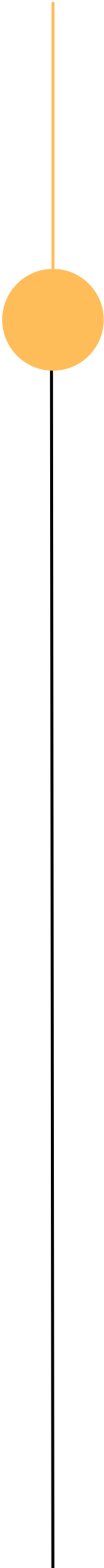
- auth
- users
- appointments
- billing
- reports
- notifications

A decorative orange circle is positioned on the left side of the page, with a thin orange vertical line extending upwards from its top and a thin black vertical line extending downwards from its bottom.

API Documentation Checklist

Before publishing the API documentation, ensure every endpoint includes:

- Endpoint URL
- HTTP Method
- Purpose/Description
- Authentication requirements
- Headers
- Path parameters
- Query parameters
- Request body schema
- Validation rules
- Success response
- Error responses
- HTTP status codes
- Sample request
- Sample response
- Rate limits (if applicable)
- Related business rules (if applicable)



Contact:

For more details or sample documents, feel free to connect with me:

Geetanjali Dhoke Business Analyst

 LinkedIn: <https://www.linkedin.com/in/geetanjali-dhoke-51a9711a1/>

 Email: gorgeousgeetanjali15@gmail.com

Thank you !